

ONE PAGE TL;DR:

TOHSENO is a person-to-person native software network. It exists because software has become easier to generate than to keep, understand, move, or continue. Today an app is usually trapped inside a vendor account, a repository, a binary, an App Store listing, or the machine and model session that produced it. TOHSENO makes a different object primary: continuing software with an identity, source, history, authority, and path from one person's hands to another's.

The system has three centers of truth. **The Mac is the factory.** It holds the source, runs the coding intelligence, builds with Xcode, verifies the result, and installs it on a physical iPhone. **The iPhone Companion is the human authority.** It holds the Builder DeviceKey, approves exact public actions, carries private change requests, and performs the Claim ritual. **The Registry is the public witness.** It combines an immutable on-chain checkpoint with a Builder-signed catalog release and exact source bytes so strangers do not have to trust the operator's database. A link is only transport between these truths; it is never authority by itself.

A Builder can adopt an existing Xcode project without rewriting it, or create a new app through the Tohseno factory. Private work stays on the Builder's Mac. When the Builder chooses **Ship**, the Mac creates a deterministic, sanitized, buildable source snapshot. The Companion shows the exact bounded publication and approves it. The network accepts the release only when the DeviceKey signature, live BuilderAccount authority, Registry receipt, public checkpoint, signed catalog, and source digest all agree. Software is shipped exactly once. Every later public release of the same software is an **Update**. A fork starts different software with a new random ShotID and an explicit relation to the exact parent release.

The recipient does not receive a magical binary or borrow the Builder's Apple identity. Their Mac verifies the release, safely extracts the source, checks whether it is suitable for automatic building, invokes their local Xcode, signs the app with their own Apple development identity, and installs it on their own iPhone. This direct path skips App Store submission and review. It does not skip Apple's code signing, provisioning, Trust, Developer Mode, Xcode, entitlements, or operating-system security. Source can travel without allowing the transport server or recipient to impersonate the Builder.

Every public Shot opens one immutable **Claim Edition** at first Ship. Another Tohseno identity can draw one circle around the app in Companion and Claim the exact Shot release and public checkpoint it encountered. The normalized mark is committed, the Companion authorizes the action, and an additive non-transferable receipt is recorded on Robinhood Chain. A Claim is not payment, purchase, license, installation proof, transferable inventory, proof of unique humanity, or ownership of the Shot. It is a durable public sentence: *this Tohseno identity encountered this software at this point in its life*. Claiming then queues private preparation of that exact release on the person's Mac, even if the Mac is offline. Installation remains a later, separate, private truth.

Underneath the network is the **Shot**, TOHSENO's continuity primitive. A Shot has a random stable identity and an authorized, reducible history binding origin, accepted promises, concrete bodies, immutable accepted states, evidence, and change. Its implementation, model, repository, language, bundle, device, or Builder can change without silently rewriting the past. The protocol does not decide whether a rewrite is philosophically the same software or whether it is good. It makes the continuity judgment explicit, attributable, and mechanically checkable. Git still names content; Xcode still builds; Apple still signs; the chain still witnesses. TOHSENO composes those systems around a continuing software object.

The social surface follows the same restraint. **Discover** is a deterministic timeline of software births, Updates, forks, and edition closures - not posts and not an App Store ranking. **Following** is a private set synchronized between a person's Mac and Companion, with no public graph or follower count. **Updates** is a private high-signal inbox for facts that affect the person's own apps, Claims, forks, approvals, or preparations. Public activity is deliberately narrow; source, prompts, local paths, Apple credentials, device identity, installs, private intentions, and private evolution remain outside it.

As of this edition, generation 0.8.0 is the client-trusted active Registry generation. The additive Claims contract has separately threshold-verified activation and production read indexing. The 1.2.0 release candidate is Developer ID signed, Apple-notarized, stapled, digest-pinned, and available only on the labeled candidate channel. Registry and Claims re-layers, Claims writes, and stable promotion remain dark until the owner-attended physical sequence proves one real Ship and edition, a second identity's Claim, offline-Mac preparation, recipient-signed iPhone installation, a later Update preserving the Claim, private Follow reconciliation, live receipts, and exactly one Ship. TOHSENO's central discipline is simple: missing evidence never becomes a success claim.

TOHSENO: Person-to-Person Native Software

A protocol, Mac factory, iPhone authority, and public witness for software people can carry forward

Author: JP Franetovic

Affiliation: Anky, Inc.

Edition: Third working-paper edition

Date: August 31, 2026

Status: Pre-1.2 working paper; protocol-exact, release-state bounded

Abstract

Software distribution usually moves an artifact while leaving continuity, authority, source custody, and future change to social convention. Central stores can distribute binaries efficiently, repositories can preserve exact content, package systems can reproduce artifacts, and signatures can attest an actor. None of those mechanisms alone defines one continuing software object that can change bodies, move between people, remain privately evolvable, and still expose a narrow public history.

TOHSENO proposes a person-to-person native software network built around such an object: the **Shot**. A Shot is a random stable identity plus an authorized, deterministically reducible history. The Apple implementation gives that protocol a concrete social and technical form. A Builder's Mac owns execution truth. A paired iPhone Companion owns human authorization truth. The generation-0.8 ShotRegistry and a signed off-chain catalog jointly own public network truth. A recipient verifies exact source, builds locally with Xcode, signs with their own Apple identity, and installs on their own iPhone. App Store submission and review are absent from this direct path; Apple's signing and operating-system boundaries remain.

The network names four public events with care. A Shot is **Shipped** once. Later public releases are **Updates**. A **Fork** begins a different Shot from one exact parent release. A **Claim** is one identity's non-transferable, public receipt for encountering a Shot at an exact release and checkpoint. Claim is deliberately separated from payment, licensing, ownership, and installation. Discover, private Following, and private Updates make software history visible without constructing an attention market.

This edition presents TOHSENO as the system it has become: continuity protocol, native Mac factory, Companion authority, deterministic public-source distribution, public witness, and restrained social network. It also preserves the limits that make the claims defensible. A signature does not prove wisdom. A checkpoint does not contain source. A Claim does not prove installation. A build does not prove a physical device accepted it. A deployed contract or notarized artifact does not activate a release. Each truth has an evidence boundary, and the product fails closed when those boundaries do not agree.

1. What changed

The prior whitepaper centered a protocol question:

What makes a materially changed implementation a later state of the same software object?

That question remains foundational, but the repository has answered a second one:

How can that software move from one person to another without surrendering its source, its history, the Builder's authority, the recipient's Apple identity, or the ability to change it again?

The answer changed TOHSENO's center of gravity. The earlier paper described the Shot, neutral lineage, Apple-oriented factory, optional Witness, and an incomplete public lifecycle. Since the commit audited by that edition, the implementation has moved through more than one hundred commits and a broad product reconstruction. The meaningful changes are not simply additional features.

First, `Tohseno.app` became the primary product. It is a native SwiftUI Mac application over the one existing Rust

service and factory. Browser Studio and the CLI remain support, recovery, and automation surfaces; neither is a second implementation engine.

Second, an ordinary existing Xcode project became a first-class beginning. Tohseno can adopt a project without writing into it, fabricating a prior Shot, or claiming historical lineage that was never recorded. Contact with working software can now precede a blank creation prompt. Generated apps remain supported, but generation is one source of software rather than the definition of the system.

Third, the iPhone Companion changed from an optional remote control into the human authority for the public network. It holds the Builder DeviceKey, approves exact Ship and Update actions, authorizes profiles, performs Claims, carries durable private evolution requests, and remains cryptographically separate from Apple signing identity, pairing identity, recovery authority, and app InstallationKeys.

Fourth, the active generation-0.8 Registry was connected to a signed source catalog and recipient-local build path. A public checkpoint alone intentionally contains almost nothing. A catalog release alone is an off-chain assertion. Exact source bytes alone carry no Builder authority. Together, with live contract state and canonical transaction evidence, they make a release independently verifiable.

Fifth, public time gained a grammar: **Ship once, Update forever**. A fork is not another update. Installation is not public acquisition. Claim became the durable encounter primitive, with one immutable edition per Shot and one non-transferable receipt per claiming Tohseno identity.

Finally, the Registry became a living software world rather than a catalog grid. Discover records births and changes. Following stays private. Updates stays personal and high-signal. The design deliberately refuses public follower counts, engagement rankings, a wallet-connect ceremony, and transferable Claim inventory.

This whitepaper therefore no longer presents Apple as an example attached to a general protocol. It presents the protocol and the Apple network as two layers of one current system. The normative encodings still live in `protocol/`; the network decisions add product, transport, evidence, and release law without rewriting the frozen protocol or the deployed generation-0.8 ABI.

2. The proposition

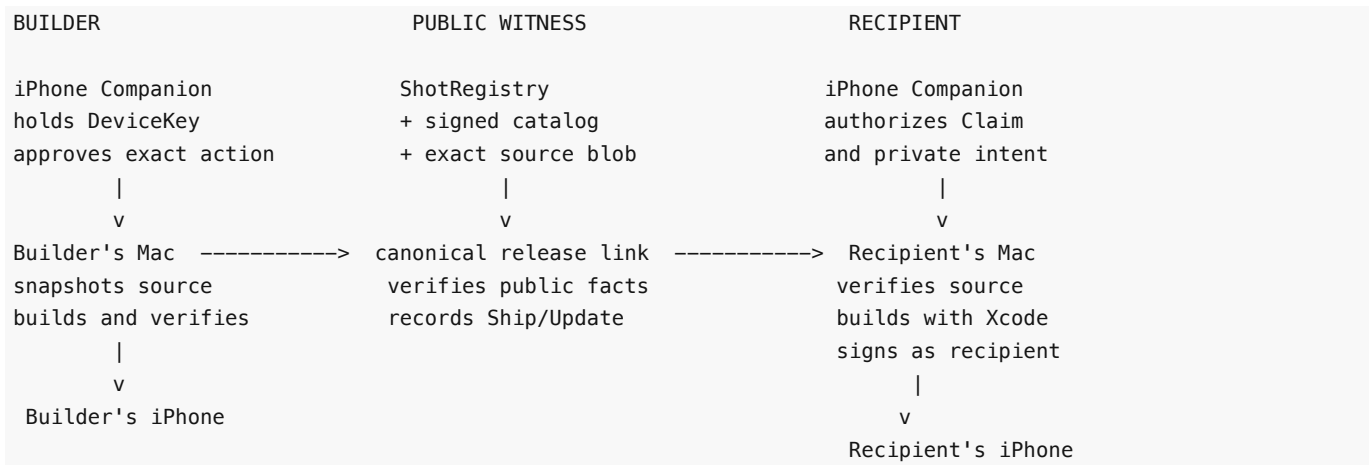
TOHSENO begins from a practical observation: generative intelligence lowers the cost of producing source, but it does not automatically create custody, continuity, provenance, distribution, or a durable relationship between a person and the software in use.

A prompt can produce an app. It cannot by itself answer:

- Where does the exact source live after the model session ends?
- Who may authorize the next accepted change?
- Which prior state was that change based on?
- Can another person obtain and inspect the exact software?
- Can that person build it without borrowing the original Builder's credentials?
- Can a radical rewrite remain the same software without pretending its bytes are unchanged?
- Can a fork acknowledge its origin without inheriting authority?
- Can an encounter matter publicly without becoming a purchase or a tradable asset?

TOHSENO's proposition is that native software can move person to person as **verifiable source with continuity**, while each participant retains their own execution and platform authority.

The core path is:



No arrow in this diagram transfers a private key. No server compiles on behalf of the recipient. No chain stores source. No Claim proves that an iPhone install occurred. Each boundary transports only what the next boundary can verify.

3. Three truths and the systems between them

The architecture is easier to understand when its authorities are kept separate.

Truth	Authority	What it can establish	What it cannot establish
Human authorization	iPhone Companion and its Builder DeviceKey	The authorized human approved one exact bounded Tohseno action	That source built, a transaction landed, or an iPhone installed the app
Execution	The Mac, Xcode, deterministic gates, and physical device observation	Which source was observed, built, locally signed, installed, and verified	That a public release was authorized or canonically witnessed
Public network	Active ShotRegistry state, canonical receipts, signed catalog, and exact source bytes	Which Builder-authorized release is discoverable at which checkpoint	Private intentions, device identity, installation, or semantic quality

Several supporting systems connect these authorities without replacing them.

The **content-blind relay** transports signed, end-to-end encrypted Companion envelopes. It can retain opaque mail while a Mac sleeps. It cannot read intentions, sign a release, run a model, build source, or create public authority.

The **catalog service** stages source, indexes public evidence, serves immutable blobs, and orchestrates tightly constrained transaction jobs. Its database is an index. Security-sensitive clients check current chain state and signatures again.

The **Registry contracts** establish BuilderAccount authority and public Shot checkpoints. They do not store app code, artifact digests, intentions, prompts, Apple credentials, installation identity, or private lineage.

The **Apple platform** remains the final code-signing and device-execution boundary. Tohseno does not collect Apple credentials or substitute a Tohseno signature for Apple's.

The **coding harness** proposes source changes. It is not the Shot, Builder, Verifier, or final acceptance authority. A successful model process is only a candidate outcome until deterministic gates pass.

This separation is not incidental complexity. It is the mechanism that allows source to move while authority, execution, and private life remain bounded.

4. The Mac is the factory

4.1 One native product over one persistent factory

Tohseno.app is a normal macOS application, not a browser wrapper. It owns navigation, windows, file selection, drag and drop, keyboard interaction, restoration, menus, accessibility, device guidance, and presentation. Behind it, the existing Rust Local Workspace Service remains the single admission and execution boundary.

The service is persistent. Closing the Mac window does not cancel an admitted request. Every command is journaled before semantic execution. Stable command, app, Shot, project, and execution identities make retries idempotent. A service restart reconstructs accepted work from durable state rather than from UI memory.

The native app is a client. It does not implement another Shot reducer, build system, signing path, or file mutator in Swift. Browser Studio and the CLI converge on the same application service. This one-factory rule matters because a simple interface is trustworthy only if it is a projection of the same state that recovery and automation use.

4.2 Adopt existing software or create it

The primary starting point is contact with software that already exists. A person selects an .xcodeproj or .xcworkspace. Tohseno inspects the container, scheme, bundle settings, deployment target, Git state, relevant repository instructions, available harness, and device/build observations. It writes no metadata into the selected repository merely because it was adopted.

Adoption creates a private, stable project relationship. It is not a claim that Tohseno existed in the project's past. Moving the folder does not silently create a new identity, and losing the folder does not fabricate one. When the owner initializes the public-network path, the observed source becomes a truthful new Tohseno root. Earlier Git, App Store, and product history can remain human context, but it is not rewritten as protocol ancestry.

Creation remains a complete second path. A person can write one intention, optionally name the app, attach up to eight validated PNG or JPEG references, choose or accept an intelligence route, and send. Plain Return sends from a focused composer; Shift-Return inserts a line. There is no Conception phase or planning round trip in front of the build. The engine composes and accepts the initial Genome in the one bounded birth run.

4.3 Change from the phone, execute on the Mac

For living projects, the ordinary loop begins on the iPhone: open the app in Companion, write what should change, and press **Evolve App**. The request binds the exact observed base. The phone signs it, encrypts it to the Mac, stores it durably, and can retry through the relay. The Mac authenticates the command, checks capability and revocation, rejects replay, verifies the exact base again, journals the command, and only then runs the configured harness in the exact source root.

If the base changed before admission, the request is stale. Tohseno never silently rebases it. If the phone closes after durable submission, the Mac can continue. If the Mac is offline, the phone's outbox waits. If the relay repeats an envelope, layered idempotency returns the same result rather than evolving twice.

The phone is therefore the place where a human notices and requests change. The Mac remains the place where source is actually changed.

4.4 Bounded intelligence and deterministic completion

TOHSENO can use an authenticated local coding harness, an owner-approved custom executable, an explicitly consented loopback model endpoint, or optional managed inference. Local and bring-your-own execution are not gated by a Tohseno subscription or managed balance. Managed execution is separately consented, priced, reserved against an append-only balance, and hard-capped before source is sent.

One request permits one implementation harness invocation and, only for a concrete code or build defect, at most one focused repair. The two share one wall-clock budget. Device, signing, provisioning, network, lineage, and protocol failures never invoke intelligence as a generic retry mechanism.

A harness exit is not completion. The factory performs deterministic source, build, test, code-signing, installation, launch, and acceptance gates. A missing iPhone becomes **Ready for your iPhone** rather than success. Reconnecting the intended physical device resumes delivery without another model call. **Installed** requires the supported CoreDevice installation to succeed and the exact bundle to appear in that phone's inventory.

The selected-app workspace exposes the useful human path: **Build / App / Source**. It can show bounded changed files, semantic activity, an honest non-interactive Simulator capture, the cable stage, and a deliberate Details receipt. It does not restore the deleted execution dashboard, raw harness output, internal phases, prompts, or protocol controls to the normal path.

5. The Companion is the human authority

The Companion is a native iPhone application installed and paired during Tohseno onboarding. It is both a private interface to a person's Mac and the approval surface for bounded public actions. Those roles share a product but not a single undifferentiated key.

The private Mac-Companion channel uses pairing identity, authenticated key agreement, revocable capabilities, signed commands, end-to-end encryption, durable outboxes, acknowledgements, replay protection, and bounded retention. It carries private app snapshots, intentions, status, Following, and Updates. The relay cannot decrypt that traffic.

Public Builder authority uses a protocol-compatible P-256 **Builder DeviceKey**. Its private scalar remains on the iPhone under the strongest compatible this-device-only, non-exportable Keychain or Secure Enclave mechanism. The Mac sends structured action facts, not an opaque digest. Companion validates the closed payload, recomputes canonical digests, presents a bounded human summary, and signs only the exact approval.

For a first Ship, one approval may cover only the exact catalog release, the exact Registry registration, the exact BuilderAccount bootstrap facts when needed, and the immutable Claim Edition policy. A later Update approval may cover only that release and its exact AppendCheckpoint action. Companion does not grant future publication authority.

The Builder DeviceKey is not an Apple signing certificate. The Mac never exports an Apple private key to Companion. It is also distinct from the recoverable private pairing identity, the Builder recovery authority, and each generated app's InstallationKey. Conflating those roles would allow compromise in one domain to impersonate another.

Companion's public Claim action is expressive but still exact. A person draws one forgiving circle around an app. The gesture is normalized to a unit canvas, resampled to exactly 64 points, encoded in a fixed-width closed format, and committed with SHA-256. Timing, pressure, velocity, device motion, and biometric inference are not retained. The gesture is not entropy and not authentication; the DeviceKey remains the authorization proof.

The result is a human authority surface that says more than "a key signed" but less than "a human can never be compromised." Tohseno can prove which key, payload, digest, account state, and receipt agreed. It cannot prove attention, wisdom, legal title, or freedom from coercion.

6. Public source as a deliberate boundary

Person-to-person distribution begins by treating source publication as a security and privacy event, not as a casual archive upload.

The Mac constructs a deterministic sanitized snapshot in a private temporary directory without changing the working source. It excludes version-control internals, build products, DerivedData, user settings, caches, environment files, private Tohseno state, logs, provisioning material, signing material, and known secrets. High-confidence secret findings fail closed and identify paths without disclosing contents. `.gitignore` is useful repository policy, but it is not the publication security boundary.

Snapshot paths are normalized, sorted, relative, collision-checked, and bounded. Symlinks, hard links, special files, traversal, ambiguous archives, and oversized trees are rejected. Deterministic metadata makes the same source produce the same artifact bytes. The catalog binds both the artifact SHA-256 and the protocol source-tree commitment.

Internet source is executable input. A narrow **Green Install Profile** permits automatic building only for ordinary native iOS source with pinned dependencies and without arbitrary Run Script phases, custom executables, unsafe build rules, compiler or package plugins, unsupported entitlements, or archive ambiguity. A non-Green release is not called malicious; it is labeled **Requires review on your Mac** with reasons before any `xcodebuild` invocation. Unsupported behavior fails before build.

The recipient extracts into a new root, recomputes the source-tree commitment, and verifies the public release evidence independently. Xcode uses the recipient's local development team. If the original bundle identifier cannot be registered, Tohseno derives a stable recipient-local namespace through build-setting overrides rather than silently rewriting downloaded source. Capabilities that cannot survive recipient signing fail with their exact reason.

This path changes the usual distribution object. The recipient does not merely trust a Builder-signed binary, and the Builder does not need the recipient's Apple credentials. The transmitted object is exact, authorized, buildable source plus enough evidence to decide what it is.

7. The Registry is a shared public witness

The Registry is not one database and not one contract. It is the agreement of several independently checkable facts.

7.1 BuilderAccount authority

A Tohseno BuilderID is the address of a generation-0.8 BuilderAccount on Robinhood Chain. Its active P-256 DeviceKeys are contract authority. The address is deterministically predicted from the approved factory generation and the initial DeviceKey, but prediction alone is not deployment or live authority. Security-sensitive clients verify the threshold-signed generation activation, runtime code, constructor state, current keys, nonces, and receipts.

The account is deployed lazily for a person's first public Tohseno action - Ship or Claim. The constrained relayer may submit the exact factory call, but it never receives the DeviceKey and cannot become a generic wallet.

7.2 Public checkpoints

The active ShotRegistry stores a random ShotID, controller, current head, checkpoint sequence, and associated contract state. First publication uses commit/reveal with a fresh privately persisted salt and the exact minimum delay. Checkpoint sequence begins at one even when the private app already has many Versions. Later Updates append exactly one checkpoint. Registry transfer changes controller while preserving head and checkpoint count.

Every head is the SHA-256 commitment of a closed `tohseno.public-checkpoint/1` object. That object intentionally excludes source, artifact digest, intention, Genome, ExpressionID, VersionID, private lineage, installation facts, controller, free text, and hashes derived from private material. Its narrowness prevents a public witness from becoming a covert publication of private ancestry.

A checkpoint proves very little alone. Its authorization comes from the paired Registry action, live ERC-1271 decision, canonical receipt, and trusted active generation.

7.3 Signed catalog releases

The off-chain `tohseno.catalog-release/1` object binds the public checkpoint to the software people can actually fetch. It includes the active generation and witness, ShotID, BuilderID, immutable release ID, time, display metadata, source artifact digest and length, source-tree commitment, Xcode container and scheme, original

bundle identifier, minimum iOS version, device family, dependency locks, safety classification, install and fork declarations, optional exact parent release, expected checkpoint sequence, and public-checkpoint digest.

The Builder DeviceKey signs that complete canonical object. It contains no private intention, prompt, reference, absolute source path, environment, pairing capability, Apple credential, profile, certificate, private key, or installation fact.

A public release becomes discoverable only when these agree:

1. the catalog's Builder DeviceKey signature;
2. live BuilderAccount authority;
3. the signed active generation, chain, and Registry;
4. canonical transaction and block evidence;
5. the Registry's current head and checkpoint sequence;
6. the manifest's public-checkpoint digest; and
7. the staged source bytes, artifact digest, and source-tree commitment.

The chain proves a Builder-controlled checkpoint. The signed catalog binds that checkpoint to an exact release. Content addressing proves the fetched bytes. None can safely substitute for the other two.

7.4 Links, aliases, and profiles

The canonical software link uses immutable ShotID and release identity. Human-readable Builder slugs and global aliases are routing conveniences, not ownership. A signed Builder profile may bind BuilderID, display name, granted handle, avatar commitment, external attestations, and update time. External identity can attest that an account maps to a BuilderID; it cannot replace DeviceKey-to-BuilderAccount authority.

A link may open the Mac or Companion at an exact Shot and release. The receiving client resolves every security-sensitive fact again. Deep-link parameters are references, never trusted release records.

8. The public grammar: Ship, Update, Fork, Claim

8.1 Ship once

A private or local candidate becomes a discoverable network Shot exactly once. That birth is **Ship**. Its `shipped_at` derives from canonical first-registration evidence, not from application-server time. First Ship also opens the Shot's one immutable Claim Edition.

This matters because birth cannot happen repeatedly without erasing the meaning of history. Marketing versions, Git tags, App Store builds, and private Versions may be numerous before public birth. The Registry still records exactly one Ship.

8.2 Update forever

Every accepted later public release of the same Shot is an **Update**. The developer operation may remain `tohseno deploy`, but the resulting fact is not another shipment. An Update binds a new source release and public checkpoint while preserving the ShotID and Claim Edition.

Updates do not reset Claim supply, reopen a timed edition, erase earlier releases, or invalidate existing Claims. A person who claimed an earlier release retains that exact encounter even when the Shot changes later.

8.3 Fork explicitly

A Fork begins with one immutable parent release, then creates a different random ShotID and local project identity. The recipient receives visible owner source, not the parent's authority or private history. The relationship binds child ShotID, parent ShotID, and parent release digest. The child stays private until its Builder ships it once and opens its own edition.

Forking therefore says both "this came from there" and "this is now different software." It avoids the two common fictions: pretending the fork has no origin, or pretending the fork can act as its parent.

8.4 Claim an encounter

A Claim says:

This Tohseno identity encountered and claimed this Shot at this exact point in its life.

At first Ship, the Builder selects one immutable edition policy: open, limited, timed, or limited-and-timed. Zero supply means unlimited and zero close time means never. There are no prices, quantities per person, auctions, tiers, allowlists, paid mints, per-release editions, or later administration.

One Tohseno account can Claim a Shot once. The action binds the Claims contract and chain, trusted ShotRegistry, claimant account, ShotID, exact release digest, exact current public checkpoint, Claim-mark commitment, nonce, and deadline. The contract checks that the checkpoint is still current at execution. Changed heads, duplicate Claims, closed editions, exhausted supply, stale nonces, wrong domains, and invalid ERC-1271 signatures fail.

The additive TohsenoClaimsV1 contract is non-upgradeable and references the unchanged active ShotRegistry. It has no owner mint, policy editor, supply override, pause, confiscation, URI replacement, transfer, approval, or upgrade path. It records an immutable edition, one immutable Claim per account and Shot, sequential Claim numbers per Shot, and global token IDs. The receipt uses ERC-721-compatible structure with transfer and approval paths closed; it is not inventory.

A Claim is deliberately not:

- payment or purchase;
- a software license or entitlement check;
- proof that one unique human exists;
- proof that Xcode built or an iPhone installed the app;
- Builder authority or control of the Shot;
- a transferable asset, marketplace item, or scarcity claim over public source.

This negative definition is essential. Installation is too contingent and private to serve as the durable public relationship. Payment would make software encounter a commercial event. A generic wallet would merge Tohseno authority with unrelated financial identity. Claim creates a narrower public fact.

8.5 Claim, then prepare

Canonical Claim confirmation automatically persists a private intention to prepare the exact claimed release on the person's Mac. It does not ask the chain or website to build anything. If the Mac is asleep, the encrypted outbox waits. When the Mac returns, it verifies the release independently, downloads it, checks safety, and prepares it through the existing local path.

If an Update exists, the claimed release is prepared first. **Update available** is a separate fact. If Xcode, signing, a cable, Trust, Developer Mode, entitlements, or hardware prevent installation, the Claim remains valid and the failure remains truthful. Claim and Install are two different verbs because they are two different kinds of evidence.

9. A social network for software, not attention

The Registry's social shape follows public software facts rather than human posting behavior.

Discover is a deterministic, reorg-aware timeline with a closed event set: `shot.shipped`, `shot.updated`, `shot.forked`, and `claim.edition_closed`. Events are ordered by canonical block and receipt facts. Individual Claims do not flood the feed; counts and edition closure summarize them.

Following filters public Builder events by exact BuilderID. The set is private to the person's Mac and paired Companion. Follow and Unfollow are idempotent, work while either peer is offline, reconcile in both directions, and survive handle changes. There is no public follow server, graph, count, leaderboard, popularity score, or notification-per-update economy.

Updates is a durable private inbox for facts with a direct relationship to the person: a claimed app changed, preparation became ready, a fork of their Shot shipped, their edition closed, an alias changed, publication

needs approval, or evolution completed. Stable event IDs and reconciled read state avoid restart spam. Generic Discover activity does not enter it.

A Shot page can therefore show one birth, subsequent Updates, exact fork relations, Claim count and edition state, and verifiable release evidence. A Builder page can show signed identity, attestations, shipped software, and activity without turning other people into a visible audience metric. A person's own Companion Profile can become a cabinet of claimed software: exact encounter, mark, Builder, release, and expandable public receipt.

This is a different model of software discovery. It asks what was born, what changed, where it came from, who authorized it, and which encounters mattered. It does not ask what maximizes time on a feed.

10. The Shot underneath the network

The public network depends on a deeper continuity object. Without it, Ship and Update would be labels on unrelated artifacts.

10.1 Definition

A **Shot** is a random stable ShotID plus a signed founding commitment and the authorized reducible history that begins with it. That history can contain:

- the founding commitment to exact Intention material and initial authority;
- an accepted, explicitly revisioned Genome;
- declared Expressions and immutable accepted Versions;
- VerificationResults, Evidence, and version-bound Feedback;
- Evolutionary Intents and later Evolutions;
- controller changes, availability observations, and parent relations.

The Shot is not its name, alias, folder, repository, prompt, source tree, current body, binary, bundle identifier, App Store listing, token, controller, Generator, checkpoint, or any one Version. Each may change or disappear without changing the ShotID. The ShotID alone proves almost nothing; it becomes meaningful only with a valid founding action and enough authorized lineage to reduce state.

10.2 Origin, promises, bodies, moments

The **Intention** preserves committed origin. It may include words, images, references, or other exact material descriptors. It is historical evidence, not a product brief that can be edited after implementation.

The **Genome** preserves accepted promises: purpose, intended users, essential experience, behavioral and experiential invariants, privacy principles, boundaries, non-goals, required capabilities, forbidden transformations, acceptance principles, platform commitments, and matters free to change. A new Genome becomes current only through an explicit proposal and acceptance. A Version cannot silently revise the law it claims to satisfy.

An **Expression** is one concrete body in an environment. An iPhone app and a later web implementation may be separate Expressions of one Shot if the controller explicitly admits them under accepted Genome commitments. The protocol exposes that judgment; it does not prove semantic sameness.

A **Version** is one immutable accepted state of one Expression. It binds the Expression, ordinal, current Genome, source commitment, provenance, capability graph, matching successful VerificationResult, known incompleteness, time, and optional build identity. Its VersionID is content-bound within the Shot and Expression:

```
VersionID = SHA-256(  
  "TOHSEN0-VERSION-ID-V2\0" || ShotID || ExpressionID ||  
  u64be(ordinal) || genome_digest || source_digest  
)
```

An **Evolution** retrospectively relates adjacent already accepted Versions through an earlier Evolutionary In-

tent. Version acceptance does not require history to pretend that every successful change began with perfect foresight. When Evolution is recorded, it makes the relationship and intended scope explicit.

10.3 Authorized reducible history

The neutral lineage is an append-only, single-predecessor sequence on each branch. Every action names its sequence, prior commitment, ShotID, actor, time, availability, payload, and payload digest. Its canonical action commitment is SHA-256 over RFC 8785 JSON, signed once as a P-256 prehash.

A deterministic reducer rejects gaps, changed ShotID, backward time, unknown schema members, invalid signatures, actors that do not match derived controller state, and illegal payload transitions. Competing valid successors form incompatible branches. The protocol does not silently choose whichever record arrived last as a universal head.

This construction establishes branch-local integrity, authorization, causal order, and declared state. It does not establish usefulness, originality, safety, semantic fidelity, legal ownership, independent verification, or a globally preferred branch.

10.4 Local continuity and public checkpoints are different

A full local lineage can contain private intention, private references, local source commitments, Feedback, and ancestry. Publishing its head would leak a digest path to private material. Generation 0.8 therefore uses an ancestry-free public checkpoint with its own witness-local sequence. The checkpoint is a narrow public time marker, not an exported local Version.

The signed catalog can bind an exact public release to that checkpoint without changing the frozen lineage format. Claims can bind an encounter to the release and current checkpoint through a separately activated additive contract. Product evolution is therefore additive around the protocol: public distribution and social facts do not rewrite historical byte encodings.

11. Apple distribution without pretending Apple disappeared

The current network is deliberately Apple-specific in execution. That specificity makes the claim testable.

On the Builder side, Xcode compiles and signs the Builder's local candidate for their own device. Tohseno can prove a physical install only through Apple's supported tools and exact device inventory. On publication, the network distributes source and release evidence, not the Builder's provisioning profile or signing identity.

On the recipient side, full Xcode performs a local build. The recipient selects or already has an Apple development team. Tohseno applies bounded build-setting overrides for recipient-local identity and then lets Xcode and Apple decide what can be provisioned. Trust, Developer Mode, cable reachability, device lock state, entitlement support, provisioning expiration, and signing failures remain visible states.

This direct developer path removes App Store submission and review from the transfer between two people. It does not create a universal consumer installation mechanism. Current participation requires a compatible Mac, full Xcode, an iPhone, and an Apple Account; provisioning and Apple policy still constrain how long and where a development-signed app runs. Broader Apple distribution remains Apple's domain.

The distinction matters politically and technically. "Skip App Store submission" is a bounded workflow statement. "Bypass Apple security" would be false. TOHSENO's model is interesting precisely because it works with recipient signing rather than distributing someone else's signing authority.

12. Privacy and security boundaries

TOHSENO is local-first, but local-first is not a claim that nothing becomes public. It is a rule that public facts are explicit, narrow, and authorized.

12.1 What stays private

Private by default are intentions, prompts, reference images, source paths, local working state, exact private lineage, Feedback, execution records, raw harness output, model credentials, Apple credentials, provisioning material, pairing capabilities, device names, InstallationKeys, physical install facts, private Following, and private Updates.

Generated app directories contain an integral `.tohseno/` boundary. Safe identity and integrity records may remain visible and Git-trackable. Exact intentions, references, feedback, executions, logs, and `.tohseno/private/` stay explicitly ignored. The directory is never blanket-ignored because doing so would discard durable app-local identity and history. Publishing a Git repository is never treated as consent to publish the private paths inside this boundary.

12.2 What becomes public

Ship makes the selected sanitized source snapshot, signed catalog metadata, ShotID, BuilderID, public checkpoint, canonical receipts, and bounded display metadata public. Claim makes the claimant Tohseno account-to-Shot relationship, exact encounter, Claim number, normalized-mark receipt, and chain evidence public. Companion discloses that relationship before a first Claim.

Neither act publishes the physical iPhone, Mac identity, Apple team, installation status, pairing relationship, IP address as product metadata, private intention, prompt, or local absolute path.

12.3 Fail-closed composition

TOHSENO resists substitution by requiring evidence layers to agree. A server cannot replace a source archive without breaking its digest. It cannot replace catalog facts without breaking the DeviceKey signature. It cannot invent Registry acceptance without canonical receipt and live state. It cannot sign as the Builder because it lacks the DeviceKey. A recipient cannot publish an Update as the parent merely because source was public.

The construction still depends on ordinary hard problems: endpoint compromise, key custody, malicious source, vulnerable dependencies, Xcode and platform behavior, RPC availability, relay funding, chain reorganization, denial of service, secret scanning limits, and human review. A Builder can authorize a bad release. A compromised Companion can authorize within its keys. A compromised Mac can misrepresent local execution to its owner. A Green classifier reduces automatic-build risk; it does not prove safety.

Immutable contracts and editions make defects expensive. Generation and Claims activation are therefore separate threshold-signed client trust decisions. A deployed address in configuration is never enough. Relayers are constrained transport and funding mechanisms, not authorities. On ambiguity, the relevant write or product claim remains unavailable.

13. Claims the system can and cannot make

Question	Evidence	Defensible conclusion	Not established
Are these exact source bytes?	Artifact SHA-256, deterministic archive, and source-tree commitment	Supplied bytes match the signed release	Safety, usefulness, or perpetual availability
Did the Builder authorize this release?	DeviceKey signature, live BuilderAccount authority, exact Registry action and receipt	The current authorized key approved this bounded release	Human wisdom, legal title, or uncompromised endpoints
Is this the Registry head?	Active-generation verification, canonical chain receipt, and current contract state	The checkpoint is currently accepted for the Shot	That the checkpoint contains source or private lineage

Question	Evidence	Defensible conclusion	Not established
Is this later state the same Shot?	Stable ShotID and valid authorized lineage or public checkpoint continuation	The recognized authority made an explicit continuity decision	Objective semantic sameness or quality
Did verification pass?	Matching VerificationResult and deterministic gate evidence	Every declared gate passed for that candidate	Adequacy or independence of every gate
Did an iPhone install it?	Supported install command plus exact physical device inventory	This Mac observed the exact bundle on the intended phone	Public Claim, unique humanity, or future availability
Did this identity Claim the app?	Canonical Claims receipt bound to Shot, release, checkpoint, account, and mark commitment	The account made one public encounter claim	Purchase, license, installation, Shot ownership, or personhood
Can the software continue?	Custody of authority, records, source, assets, data, dependencies, and build knowledge	Continuation is practically possible to the extent those materials suffice	Future platform access or permanent buildability

The primitive lies in this composition with constrained meanings. Strong cryptography attached to vague claims would be less useful than explicit boundaries.

14. Implementation and release state

This section is non-normative and time-bounded to August 31, 2026. Protocol, product, contract, client, and distribution versions are not interchangeable.

14.1 Implemented

The repository implements the neutral Shot lineage and conformance system; the generated Apple factory; the native Mac app; private existing-project adoption and evolution; persistent command journals; one bounded harness path; deterministic Apple build/sign/install gates; the branded Companion; authenticated encrypted pairing and relay; durable offline command delivery; generation-0.8 activation verification; deterministic safe source publication; signed catalog releases; Registry receipt verification; constrained Registry jobs; recipient download/build/sign/install/fork/refresh; Builder profiles and aliases; Claim action and mark vectors; the additive Claims contract and activation verifier; Claim receipt models; Discover, private Following, and private Updates; and end-to-end local network fixtures.

Generation 0.8.0 is the current client-trusted ShotRegistry generation on Robinhood Chain. The additive TohsenoClaimsV1 deployment has a separate threshold-signed activation bound to that exact Registry. Production reads verify activation, runtime code, Registry binding, and canonical indexing.

The current native 1.2.0 release candidate 2 was built from a clean CI-gated source commit. It is a universal Mac DMG, Developer ID signed, accepted by Apple notarization, stapled, mounted and Gatekeeper-checked, published at an immutable prerelease URL, digest-pinned, independently round-tripped from origin, and active on the website's labeled release-candidate channel.

These are real but partial facts.

14.2 Still dark or unproven

Stable 1.2.0 is not published. Registry and Claims relayers are disabled. Claims writes are disabled. The Claim index is canonically empty before the required physical walkthrough. The candidate has not completed independent clean-Mac and physical-iPhone acceptance for the full 1.2 sequence.

Activation requires one owner-attended production proof in causal order:

1. one real first Ship through normal Companion approval;
2. one immutable Claim Edition opened with that Ship;
3. a second Tohseno identity's physical Companion Claim and canonical receipt;
4. Claim while the recipient Mac is offline, followed by automatic preparation of the exact release;
5. recipient-local Xcode signing and verified physical iPhone installation;
6. one later Update that preserves the Claim and edition;
7. private Following reconciliation across Mac and Companion;
8. live public Claim receipt and metadata paths; and
9. proof that the timeline contains exactly one Ship.

No mock, simulator card, pending transaction, admin database row, source test, notarized DMG, or contract deployment can substitute for that sequence. If it fails, the candidate and evidence remain preserved, writes remain dark, and stable promotion remains closed.

14.3 Retained and deferred paths

The browser Studio, explicit recording layer, CLI, npm bootstrap, legacy cable and entitlement code, private token association, and managed Stripe/Bankr machinery remain in the repository for support, compatibility, development, or separately gated operation. They are not the primary consumer path and do not authorize release claims merely by existing in source.

No successor Registry contract generation or deployment ceremony is active on `main`. Generation 0.8 remains immutable. Claims is additive rather than a disguised Registry upgrade. Non-Apple factories, broader consumer signing distribution, independently audited immutable contracts, stronger recovery custody, and other ecosystems remain future work unless separate evidence says otherwise.

15. Non-goals

TOHSENO is not:

- an App Store clone or a universal consumer distribution service;
- an Apple security bypass, signing service, provisioning broker, or credential custodian;
- a cloud IDE, mobile source editor, autonomous deployment agent, or second coding harness;
- a blockchain source store, generic wallet, token marketplace, or financial protocol;
- a proof-of-personhood system, social popularity graph, or engagement feed;
- a replacement for Git, content addressing, reproducible builds, package managers, supply-chain attestations, Xcode, or code review;
- a machine oracle for semantic sameness, usefulness, originality, legality, or safety;
- a promise that public source will build forever or that every app can be re-signed under every Apple entitlement;
- a claim that cryptographic controller authority settles legal ownership.

TOHSENO composes existing mechanisms around a software-specific continuity and distribution object. Its novelty, if the system earns it, is not a new hash or signature curve. It is the causal architecture: one continuing Shot; source retained and deliberately published; human approval separated from machine execution; public witness separated from private life; recipient signing separated from Builder authority; and encounter separated from ownership and payment.

16. Conclusion

The first TOHSENO question was whether software could retain identity while every implementation detail changed. The Shot answered by making continuity a stable, authorized, reducible object rather than an intuition attached to a repository or brand.

The current system asks what that object is for. The answer is movement.

A Builder can begin with existing software or an intention. The Mac can change and verify it. The Companion can carry the human's authority. The Builder can Ship exact source once and publish Updates without surrendering keys. The Registry can witness a narrow public history without ingesting private lineage. Another person can Claim one encounter, bring the exact release home to their Mac, rebuild it with Xcode, sign it as themselves, install it on their iPhone, fork it honestly, and carry it forward.

That path does not abolish trust. It makes trust smaller and more inspectable. The Builder is trusted for the meaning of the software they authorize. The recipient trusts their own Mac, Xcode, review, and signing identity for execution. The network verifies the points where those worlds meet. Missing evidence stays missing.

Software can then be more than a binary delivered by an institution or a folder abandoned by a generator. It can have a birth, a changing body, a witnessed history, encounters, descendants, and people capable of continuing it.

The Mac is the factory. The Companion is the human authority. The Registry is the public witness. The next person signs for themselves.

The first Generator can disappear. The software can still move.

References

Normative and architectural sources

1. TOHSENO, [Protocol Specification and Deterministic Conformance](#).
2. TOHSENO, [ADR 0034: Tohseno Makes Native Software Person-to-Person](#).
3. TOHSENO, [ADR 0035: Claiming Software](#).
4. TOHSENO, [ADR 0033: Tohseno Maintains Living iPhone Projects](#).
5. TOHSENO, [ADR 0025: Native macOS Is the Product](#).
6. TOHSENO, [Person-to-Person Architecture, Repository State, and Privacy Boundary](#).
7. TOHSENO, [Claims Deployment and Activation and 1.2.0 Readiness Evidence](#).
8. TOHSENO, [Generation-0.8 Activation Evidence and Claims Activation Evidence](#).

Related mechanisms

9. IETF, [RFC 8785: JSON Canonicalization Scheme](#).
10. Git Project, [Git Internals](#).
11. Reproducible Builds, [Definition](#).
12. in-toto Project, [Specification](#), and SLSA, [Provenance](#).
13. W3C, [Decentralized Identifiers v1.0](#) and [Verifiable Credentials Data Model v2.0](#).
14. Ethereum Improvement Proposals, [ERC-721](#) and [ERC-5192](#).
15. Apple, [Code Signing](#) and [CFBundleIdentifier](#).

Appendix A. Compact glossary

Term	Meaning
Shot	Continuing software object with a random stable identity and authorized history
Intention	Immutable committed founding-material record
Genome	Explicitly accepted and revisioned promises and boundaries
Expression	One concrete body of a Shot in an environment
Version	One immutable accepted state of an Expression
Evolution	Retrospective authorized relation between adjacent accepted Versions
BuilderID	Generation-0.8 BuilderAccount address used as Tohseno public identity
Builder DeviceKey	Non-exportable P-256 key held by Companion for bounded public authorization
Public checkpoint	Narrow ancestry-free Registry head containing no private lineage or source digest
Catalog release	DeviceKey-signed object binding a checkpoint to exact public source and build facts
Ship	The one public birth of a Shot
Update	Any later accepted public release of that Shot
Fork	New Shot with an explicit relation to one exact parent release
Claim Edition	One immutable open, limited, timed, or limited-and-timed Claim policy fixed at first Ship
Claim	One Tohseno account's non-transferable public encounter receipt for a Shot
Claim mark	Canonical expressive circle representation whose digest is bound to a Claim
Discover	Canonical public timeline of bounded software events
Following	Private BuilderID preference set synchronized between Mac and Companion
Updates	Private relationship-backed inbox synchronized between Mac and Companion

Appendix B. Constitutional invariants

1. A ShotID is random, stable, and independent of content, name, Builder, bundle, token, or channel.
2. Accepted history is append-only; origin and accepted Versions are not edited in place.
3. Continuity is an explicit authorized judgment, not machine proof of semantic sameness.
4. A public checkpoint never imports private ancestry or hashes derived from it.
5. The Mac owns execution truth; the Companion owns bounded human authorization; the Registry and signed catalog own public network truth.
6. Public source requires exact Companion approval and deterministic sanitization.
7. A recipient independently verifies, builds, and signs with their own Apple identity.
8. A Shot Ships once. Later public releases are Updates. A Fork receives a new ShotID.
9. One Shot has one immutable Claim Edition. One Tohseno account can Claim that Shot once.
10. Claim is not payment, license, installation, ownership, personhood, or transferable inventory.
11. Following and Updates remain private; there is no public attention graph.
12. A chain deployment, database row, artifact build, signature, notarization, or source test never proves a later evidence state by implication.
13. Missing or contradictory evidence fails closed.
14. Frozen protocol encoding and the active generation-0.8 ABI are not rewritten to express new product behavior.